Term Project:
Socket Programming

Group X

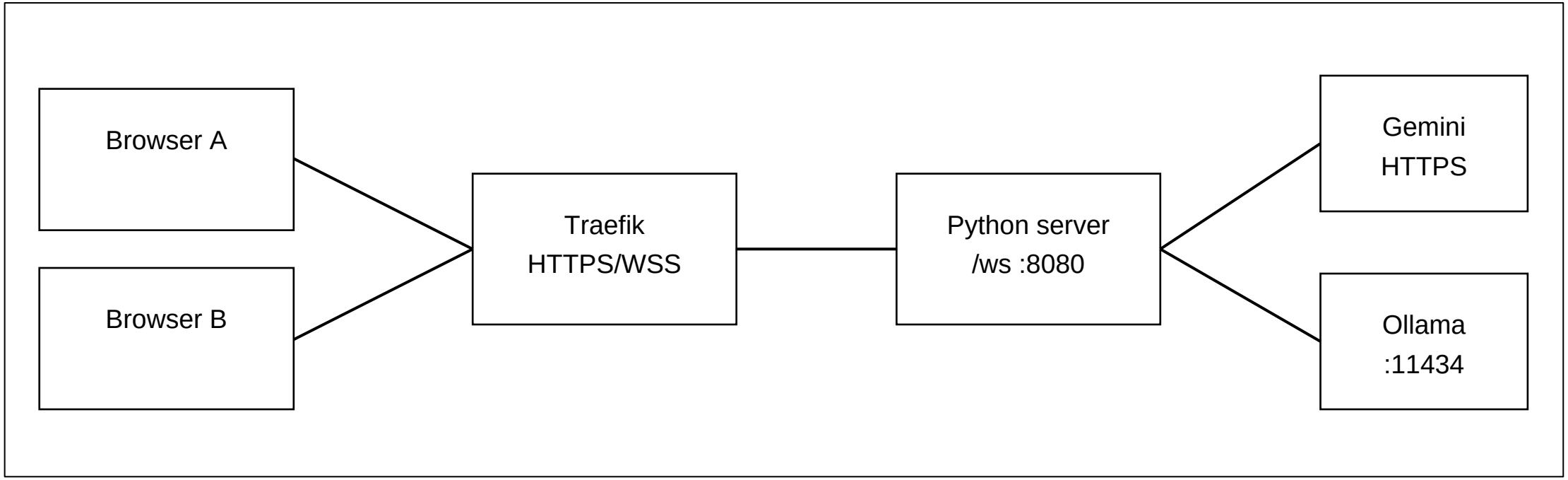
6xxxxxxxx21 Name Surname

6xxxxxxxx21 Name Surname

Part 1 System Architecture Design

This part must be fully completed; otherwise, Part 2 will not be graded.

R1 – System Architecture Design



Two laptops reach Oracle through Traefik. Chat uses one /ws connection. Each summary selects Gemini or local Qwen; Gemini failure retries locally.

R1 – System Architecture Design

- Explain the computer network protocols used in the project
  - What protocols are used
  - Why each protocol was selected
  - How each protocol is used within the project

TCP: reliable, ordered transport under HTTP and WebSocket.

HTTPS/WSS: encrypt browser traffic through Traefik on port 443.

WebSocket /ws: one persistent channel for chat, files, and receipts.

JSON: command and event format within WebSocket frames.

Internal HTTP: server to Ollama; HTTPS: server to Gemini.

SSH: GitHub Actions connects to Oracle for deployment.

R2 – The chat messages between the server and each client must be implemented using Socket Programming only

```
server.py / WebSocket transport
def create_app():
    app = web.Application()
    app['chat_server'] = ChatServer()
    app.router.add_get('/ws', ws_handler)
    return app

async def ws_handler(request):
    server = request.app['chat_server']
    socket = web.WebSocketResponse(heartbeat=30.0)
    await socket.prepare(request)
    session = UserSession(socket=socket)
    server.sessions[session.id] = session
    try:
        async for msg in socket:
            if msg.type == WSMsgType.TEXT:
                await server.dispatch(session, msg.data)
    finally:
        await server.disconnect(session)
```

Prompt: Build a single /ws endpoint; route JSON frames to a central dispatcher.

Part 2 Fundamental implementation

This part must be fully completed; otherwise, Part 3 will not be graded.

R3 – Each client must have a unique name.

```
server.py:38-49
class RegisterCmd(BaseModel):
    """Validates client registration (R3)."""
    type: Literal["register"] = "register"
    name: str

    @field_validator("name")
    @classmethod
    def validate_name(cls, value: str) -> str:
        trimmed = value.strip()
        if not (1 <= len(trimmed) <= 24) or not
NAME_PATTERN.fullmatch(trimmed):
            raise ValueError("Use 1?24 letters, numbers, spaces,
dots, underscores or hyphens.")
        return trimmed
```

Validate a 1-24 character username with Pydantic.

```
server.py:259-274
dead = [s for s in list(self.sessions.values()) if s.socket.closed]
for s in dead:
    await self.disconnect(s)

if any(s.name.casefold() == cmd.name.casefold() for s in
self.sessions.values() if s.id != session.id):
    raise ValueError("That name is already connected. Choose another.")

if len(self.sessions) >= 100:
    raise ValueError("The server is full. Try again later.")

session.name = cmd.name
await self.send(session, {
    "type": "welcome",
    "id": session.id,
    "name": session.name,
    "ai_model": AI_MODEL,
```

Reject active duplicate names, ignoring case.

R4 – Each client can see a list of names of all clients that are currently connected to the server including its own name

```
server.py:242-249
async def broadcast_presence(self):
    """Broadcast online users and active groups to all clients
(R4, R9)."""
    await self.broadcast({
        "type": "state",
        "users": [{"id": s.id, "name": s.name} for s in
self.sessions.values() if s.name],
        "groups": [self.describe_room(r) for r in
self.rooms.values() if r.kind == "group"],
    })
```

Broadcast every connected user's name in real time.

```
static/app.js:117-122
case "state":
  state.users = data.users || [];
  state.groups = data.groups || [];
  renderSidebar();
  if (state.activeRoomId) renderHeader();
  break;
```

Update the browser list when a state event arrives.

R5 – Chat between clients (both Private and Group message) must have its own chat room

```
server.py:279-292
async def handle_open_private(self, session: UserSession, data:
dict):
    """R5: Open or create an isolated 1-on-1 private room for two
clients."""
    cmd = OpenPrivateCmd(**data)
    peer = self.sessions.get(cmd.peer)
    if peer is None or peer.id == session.id:
        raise ValueError("Choose another online user.")

    members = {session.id, peer.id}
    room = next((r for r in self.rooms.values()
                 if r.kind == "private" and r.members == members),
None)
    if room is None:
        if len(self.rooms) >= 100:
            raise ValueError("Room limit reached.")
        room = ChatRoom(
```

Create or reuse a private room for two sessions.

```
server.py:234-240
async def open_room(self, session: UserSession, room: ChatRoom):
    """Deliver room metadata and message history to a client."""
    await self.send(session, {
        "type": "room",
        "room": self.describe_room(room),
        "messages": list(room.messages),
    })
```

Send room metadata and recent history to clients.

R6 – Each chat room (both Private and Group message) must include a chat box for sending text messages and chat window for displaying the sent messages and new incoming messages.

```
server.py:78-89
class MessageCmd(BaseModel):
    """Validates sending a text message (R6, R7, R11)."""
    type: Literal["message"] = "message"
    room: str
    text: str = Field(min_length=1, max_length=4000)

    @field_validator("text")
    @classmethod
    def validate_text(cls, value: str) -> str:
        trimmed = value.strip()
        if not (1 <= len(trimmed) <= 4000):
            raise ValueError("Messages must contain 1?4000
characters.")
```

Validate message text and keep 100 per room.

```
server.py:340-353
message = {
    "id": uuid.uuid4().hex,
    "room": room.id,
    "sender": session.id,
    "name": session.name,
    "timestamp": time.time(),
    "kind": "message",
    "text": cmd.text,
    "read_by": [],
    "delivered_by": [],
}

room.messages.append(message)
await self.broadcast({"type": "message", "message": message},
room.members)
```

Send each new message to its room members.

R7 – Each client can send a direct text message to any clients in the list. Only the sender and receiver can see the messages.

```
server.py:227-232
def authorized_room(self, session: UserSession, room_id: str) ->
ChatRoom:
    """Check that the room exists and the user is an authorized
member."""
    room = self.rooms.get(room_id)
    if room is None or session.id not in room.members:
        raise ValueError("Join this room before using it.")
    return room
```

Check room membership before accepting a message.

```
server.py:347-353
    "text": cmd.text,
    "read_by": [],
    "delivered_by": [],
}

room.messages.append(message)
await self.broadcast({"type": "message", "message": message},
room.members)
```

Deliver a private message only to its two peers.

R8 – Each client can create a chat group(s), which initially includes only themselves as a member.

```
server.py:58-69
class CreateGroupCmd(BaseModel):
    """Validates creating a chat group (R8)."""
    type: Literal["create_group"] = "create_group"
    name: str = Field(min_length=1, max_length=40)

    @field_validator("name")
    @classmethod
    def validate_name(cls, value: str) -> str:
        trimmed = value.strip()
        if not (1 <= len(trimmed) <= 40):
            raise ValueError("Group names must contain 1?40
characters.")
        return trimmed
```

Validate the new group's name and length.

```
server.py:303-318
async def handle_create_group(self, session: UserSession, data: dict):
    """R8: Create a chat group with only the creator as member."""
    cmd = CreateGroupCmd(**data)
    if len(self.rooms) >= 100:
        raise ValueError("Room limit reached.")

    room = ChatRoom(
        id=uuid.uuid4().hex,
        kind="group",
        name=cmd.name,
        members={session.id},
        names={session.id: session.name}
    )
    self.rooms[room.id] = room
    await self.open_room(session, room)
    await self.broadcast_presence()  # R9: everyone sees new group and
members
```

Start each group with only its creator inside.

R9 – Each client can see a list of all existing chat groups (with the member list) created by any clients.

```
server.py:215-225
def describe_room(self, room: ChatRoom) -> dict:
    """Helper to serialize room metadata for clients."""
    return {
        "id": room.id,
        "kind": room.kind,
        "name": room.name,
        "members": [
            {"id": mid, "name": room.names.get(mid, "")}
            for mid in sorted(room.members)
        ],
    }
```

Include every group's current member names.

```
static/app.js:230-239
for (const g of state.groups) {
  const joined = g.members.some((m) => m.id === state.me?.id);
  const li = document.createElement("li");
  const row = document.createElement("div");
  row.className = `group-row${g.id === state.activeRoomId ? "
active" : ""}`;
  row.title = g.members.length ? `Members: ${g.members.map((m)
=> m.name).join(", ")}` : "No members yet";

  const label = document.createElement("span");
  label.textContent = `# ${g.name} (${g.members.length})`;
  row.appendChild(label);
```

Show group members when hovering over its row.

R10 – Clients can choose to join a group chat only by themselves; they are not added automatically by the group creator.

```
server.py:72-76
class JoinGroupCmd(BaseModel):
    """Validates joining an existing group (R10)."""
    type: Literal["join_group"] = "join_group"
    room: str
```

```
server.py:320-330
async def handle_join_group(self, session: UserSession, data:
dict):
    """R10: Voluntary self-join group chat."""
    cmd = JoinGroupCmd(**data)
    room = self.rooms.get(cmd.room)
    if room is None or room.kind != "group":
        raise ValueError("This group does not exist.")

    room.members.add(session.id)
    room.names[session.id] = session.name
    await self.open_room(session, room)
    await self.broadcast_presence()  # R9: updated member list
broadcast
```

Require an explicit Join command from the user.

Add only the requesting session to the group.

R11 – Each client can send a text message to the group that they joined. Only the members of the chat group can see the messages.

```
server.py:227-232
def authorized_room(self, session: UserSession, room_id: str) ->
ChatRoom:
    """Check that the room exists and the user is an authorized
member."""
    room = self.rooms.get(room_id)
    if room is None or session.id not in room.members:
        raise ValueError("Join this room before using it.")
    return room
```

Reject text from users who have not joined.

```
server.py:342-353
    "room": room.id,
    "sender": session.id,
    "name": session.name,
    "timestamp": time.time(),
    "kind": "message",
    "text": cmd.text,
    "read_by": [],
    "delivered_by": [],
}

room.messages.append(message)
await self.broadcast({"type": "message", "message": message},
room.members)
```

Broadcast group messages to members only.

Part 3 Special Points

At least one of the features in the project must be an AI feature, such as speech-to-text, text summarization, sentiment analysis, a chatbot, etc., to demonstrate the integration of AI technologies into your system.

Special Point 1: AI chat summaries

AI models: Gemini 3.5 Flash-Lite; Qwen2.5 0.5B (Ollama)

Model refs: ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite | ollama.com/library/qwen2.5:0.5b

```
server.py:138-142
class SummarizeCmd(BaseModel):
    """Validates requesting AI room summary (Special 1)."""
    type: Literal["summarize"] = "summarize"
    room: str
    provider: Literal["gemini", "ollama"] = "gemini"
```

```
server.py:435-439,471-477
backends = ("gemini", "ollama") if provider == "gemini" and
GEMINI_API_KEY else ("ollama",)
for backend in backends:
    model = GEMINI_MODEL if backend == "gemini" else OLLAMA_MODEL
    try:
        if backend == "gemini":
# ...
    await self.send(session, {"type": "summary", "room": room_id,
                              "text": text[:6000], "model": model})
    break
except Exception:
    if backend == "ollama":
        raise
    logging.warning("Gemini summary failed; retrying with Ollama",
exc_info=True)
```

Build a room summary with Gemini or Qwen.

If Gemini fails, retry with local Qwen.

Special Point 2: WebSocket file sharing

```
server.py:93-105
class FileCmd(BaseModel):
    """Validates Base64 file payload through WebSocket (Special
2)."""
    type: Literal["file"] = "file"
    room: str
    filename: str
    data: str
    file_bytes: int = 0

    @field_validator("filename")
    @classmethod
    def validate_filename(cls, value: str) -> str:
        if not (1 <= len(value) <= 120) or "/" in value or "\\"
in value or any(ord(c) < 32 for c in value):
            raise ValueError("Invalid file name.")
```

Validate Base64 contents and the 256 KiB limit.

```
server.py:363-378

if room.kind == "private" and not room.members <= self.sessions.keys():
    raise ValueError("This user disconnected. Open a new chat when they
return.")

message = {
    "id": uuid.uuid4().hex,
    "room": room.id,
    "sender": session.id,
    "name": session.name,
    "timestamp": time.time(),
    "kind": "file",
    "filename": cmd.filename,
    "data": cmd.data,
    "bytes": cmd.file_bytes,
    "read_by": [],
    "delivered_by": [],
```

Send the file in WebSocket frames to members.

Special Point 3: Delivery and read receipts

```
server.py:124-135
class ReceiptCmd(BaseModel):
    """Validates delivery and read receipt status updates
(Special 3)."""
    type: Literal["read", "received"]
    room: str
    ids: list[str] = Field(max_length=100)

    @field_validator("ids")
    @classmethod
    def validate_ids(cls, value: list[str]) -> list[str]:
        if any(not isinstance(i, str) for i in value):
            raise ValueError("Invalid read receipt.")
        return value
```

Track received and read message IDs.

```
server.py:384-401
async def handle_receipt(self, session: UserSession, data: dict):
    """Special 3: Update delivery and read receipts for room messages."""
    cmd = ReceiptCmd(**data)
    room = self.authorized_room(session, cmd.room)
    receipt_field = "read_by" if cmd.type == "read" else "delivered_by"

    for message in room.messages:
        if (message["id"] in cmd.ids and session.id in room.members
                and session.id != message["sender"]
                and session.id not in message[receipt_field]):
            message[receipt_field].append(session.id)
            await self.broadcast({
                "type": "receipt",
                "room": room.id,
                "id": message["id"],
                "read_by": message["read_by"],
                "delivered_by": message["delivered_by"],
            }, room.members)
```

Broadcast receipt updates to room members.

Special Point 4: Oracle cloud deployment

```
compose.oracle.yml:6-16
networks:
  - default
  - proxy
labels:
  traefik.enable: "true"
  traefik.docker.network: eventory-production_default
  traefik.http.services.relay.loadbalancer.server.port: "8080"
  traefik.http.routers.relay.rule:
Host(`${APP_DOMAIN:-relay.161.118.238.148.sslip.io}`)
  traefik.http.routers.relay.entrypoints: websecure
  traefik.http.routers.relay.tls: "true"
  traefik.http.routers.relay.tls.certresolver: letsencrypt
```

Route the app through existing Traefik TLS.

```
scripts/deploy.sh:17-21,28-29
printf '%s' "$registry_token" | docker login ghcr.io -u Quackudy
--password-stdin >/dev/null
unset registry_token
compose=(docker compose -f compose.yml -f compose.oracle.yml)
previous=$(cat .image 2>/dev/null || true)
export APP_IMAGE="$image"
# ...
  exit 1
fi
```

Build, push, SSH deploy, and check health.